Studio Nx

백엔드 280 — 클립 · 연결 라우터

2026-10-19

백엔드 280 — 클립 · 연결 라우터 (api/v1/clips.py · api/v1/connections.py)

항목	내용
파일	`studionx/backend/app/api/v1/clips.py`, `studionx/backend/app/api/v1/connections.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립과 트랙 간 연결의 HTTP 라우터 (PRD 02 FR-2196, FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 279 패턴 서비스, 다음 281 패턴 라우터

1. 하는 일

클립과 연결의 HTTP 길을 두 파일에 나누어 둡니다. `app/api/v1/router.py` 에서 두 라우터를 접두사 없이 붙이므로, 화면은 `/api/v1/tracks/{id}/clips` 같은 경로로 부릅니다. 라우터는 요청 모양과 상태 코드만 정하고, 권한과 404 그리고 고리 검사는 서비스(277, 278)에 그대로 넘깁니다.

메서드와 경로	하는 일	상태 코드
`GET /tracks/{track_id}/clips`	트랙에 놓인 클립 목록을 start_tick 순으로 읽기	200
`POST /tracks/{track_id}/clips`	트랙에 클립 하나 놓기	201
`PATCH /clips/{clip_id}`	클립의 위치, 길이, 페이드, 워프 등 수정	200
`DELETE /clips/{clip_id}`	클립 삭제 (가리키던 패턴은 남습니다)	204
`GET /tracks/{track_id}/connection`	이 트랙에서 나가는 연결 읽기	200, 없으면 404
`PUT /tracks/{track_id}/connection`	연결을 보낸 값으로 통째로 맞추기	200
`DELETE /tracks/{track_id}/connection`	연결 끄기 (이미 없어도 통과합니다)	204

```
clips.py         GET, POST /tracks/{id}/clips — clip_service.list_clips, create_clip
                 PATCH, DELETE /clips/{id}    — clip_service.update_clip, delete_clip

connections.py   GET, PUT, DELETE /tracks/{id}/connection — connection_service
                 PUT 은 새로 만들거나 통째로 덮어쓰고, DELETE 는 연결을 끕니다
```

2. 코드

```
@router.post(
    "/tracks/{track_id}/clips",
    response_model=ClipResponse,
    status_code=status.HTTP_201_CREATED,
    responses=WRITE_RESPONSES,
)
def create_clip(
    track_id: Annotated[int, Path(ge=1)],
    req: ClipCreateRequest,
    db: Db,
    current_user: CurrentUser,
):
```

```
    """트랙에 클립을 놓는다 (FR-2196) — OWNER·EDITOR 만."""
    clip = clip_service.create_clip(db, current_user, track_id, req)
    return ClipResponse.model_validate(clip)


@router.delete(
    "/clips/{clip_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        401: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        **NOT_FOUND_404,
    },
)
def delete_clip(
    clip_id: Annotated[int, Path(ge=1)],
    db: Db,
    current_user: CurrentUser,
):
    """클립 삭제 (FR-2196) — OWNER·EDITOR 만."""
    clip_service.delete_clip(db, current_user, clip_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

연결 쪽은 PUT 이 저장, DELETE 가 끄기를 담당합니다.

```
@router.put(
    "/tracks/{track_id}/connection",
    response_model=TrackConnectionResponse,
    responses=WRITE_RESPONSES,
)
def set_connection(
    track_id: Annotated[int, Path(ge=1)],
    req: TrackConnectionSetRequest,
    db: Db,
    current_user: CurrentUser,
):
    """연결을 보낸 값으로 통째로 맞춘다 (FR-2197) — OWNER·EDITOR 만."""
    connection = connection_service.set_connection(db, current_user, track_id, req)
    return TrackConnectionResponse.model_validate(connection)


@router.delete(
    "/tracks/{track_id}/connection",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        401: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        **NOT_FOUND_404,
    },
)
def delete_connection(
    track_id: Annotated[int, Path(ge=1)],
    db: Db,
    current_user: CurrentUser,
):
    """연결 끄기 (FR-2197) — OWNER·EDITOR 만. 연결이 없어도 204."""
```

```
    connection_service.delete_connection(db, current_user, track_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`Annotated[int, Path(ge=1)]`	경로의 번호는 1 이상만 받습니다. 0 이나 글자가 들어오면 422입니다
`status_code=status.HTTP_201_CREATED`	만들기 응답 상태 코드를 201 로 바꿉니다
`status.HTTP_204_NO_CONTENT`	지우기는 내용 없는 204 로 보냅니다
`response_model=ClipResponse`	서비스가 돌려준 행을 응답 스키마 칸으로만 바꿔 내보냅니다
`ClipResponse.model_validate(clip)`	DB 행을 응답 모델로 검증하면서 변환합니다
`responses=WRITE_RESPONSES`	문서에 400, 401, 403, 404 모양을 적어 둡니다
`**NOT_FOUND_404`	404 항목을 사전에 합쳐 넣습니다
`req: ClipCreateRequest`	본문을 검증해 잘못된 값은 422 로 막고 넘깁니다
`CurrentUser`, `Db`	`app/api/deps.py` 별칭입니다. 로그인 사용자와 DB 세션을 자동으로 받습니다

3. 왜 이렇게

선택	이유
두 라우터를 접두사 없이 붙임	클립은 `/tracks/{id}/clips` 와 `/clips/{id}` 로, 연결은 `/tracks/{id}/connection` 하나로 읽기 쉽습니다
라우터를 얇게 둠	권한, 404, 고리 검사는 서비스 한 곳에 있으므로 라우터는 부르기와 응답 변환만 합니다
연결 저장을 PUT 으로 둠	한 트랙에서 나가는 연결은 하나뿐이라, 새로 만들기와 고치기가 사실은 같은 동작입니다
연결 삭제를 204 로 두고 오류를 안 냄	이미 꺼진 연결을 다시 끌 때 화면이 오류로 멈추지 않습니다
클립 삭제 뒤에도 패턴을 남김	패턴은 다른 클립이 쓸 수 있기 때문입니다 (277 문서)
응답 스키마를 항상 지정	문서와 실제 응답 칸이 같아지고, 비밀 칸이 새는 것을 막습니다

4. 검증

`tests/test_clip_routes.py` 와 `tests/test_connection_routes.py` 가 이 두 파일을 직접 부릅니다. `test_clip_routes_registered` 와 `test_connection_routes_registered` 가 `app.openapi()["paths"]` 에 경로가 실렸는지 확인하고, `test_create_midi_clip_201`, `test_create_audio_clip_201`, `test_patch_clip`, `test_delete_clip` 은 201, 200, 204 상태 코드를 확인합니다. `test_put_creates_and_get_returns`, `test_put_replaces_existing`, `test_delete_turns_connection_off` 는 PUT 이 만들고 덮어쓰며 DELETE 가 끄는 동작을 확인합니다. `test_viewer_can_read_but_not_write` 와 `test_viewer_cannot_write` 는 권한, `test_needs_login` 은 로그인 없음 401 을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다

확인할 것	내용
화면 확인	타임라인에 클립을 놓고 옮기고 지우기, 연결 켜기와 끄기를 시퀀서에서 직접 확인합니다